

LAB # 04

Implementation of Multilayer Perceptron

Lab Tasks:-

1. Go to Kaggle and download a suitable binary classification dataset. Train a Multilayer Perceptron (MLP) model on the dataset to predict the target variable. After training, evaluate the performance of the model using accuracy and test it on new unseen data samples.

Code:-

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

data = pd.read_csv("diabetes.csv")

X = data.drop("Outcome", axis=1)
y = data["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = MLPClassifier(hidden_layer_sizes=(5,5), max_iter=1000)
model.fit(X_train, y_train)

print("Accuracy:", model.score(X_test, y_test))

sample = X.iloc[0].values.reshape(1, -1)
print("Prediction:", model.predict(sample))
```

Output:-

Accuracy: 0.6883116883116883
Prediction: [1]

2. Modify the MLP model by changing the number of hidden layers and neurons (for example, using one hidden layer, two hidden layers, and different numbers of neurons). Train the model using different learning rates such as 0.01 and 0.1. Plot the loss versus epochs graph for each configuration and analyze how these changes affect the convergence speed and accuracy of the model.

Code:-

```
import matplotlib.pyplot as plt
configs = [
    ((5,), 0.01),
    ((10,), 0.1),
    ((5,5), 0.01),
    ((10,10), 0.1)
]

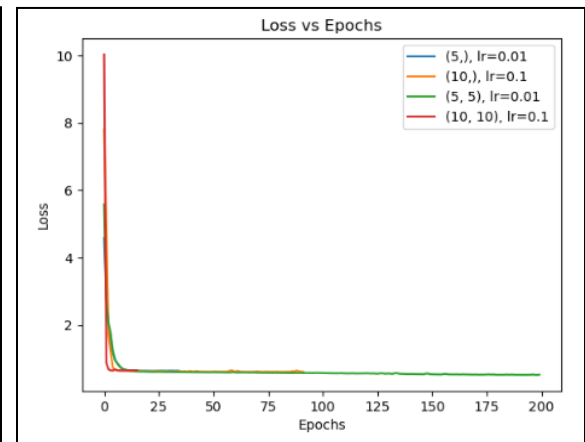
for layers, lr in configs:
    model = MLPClassifier(hidden_layer_sizes=layers,
                          learning_rate_init=lr,
                          max_iter=200)

    model.fit(X_train, y_train)

    plt.plot(model.loss_curve_, label=f"{layers}, lr={lr}")

plt.title("Loss vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

Output:-



Bonus Task:

Download a dataset containing three input features and three output classes from Kaggle or any CSV file. Implement a multi-class MLP model and train it on the dataset. Visualize the data in three dimensions and analyze how different features contribute to the classification

Code:-

```
from mpl_toolkits.mplot3d import Axes3D

data = pd.read_csv("Iris.csv")

X = data[["SepalLengthCm", "SepalWidthCm", "PetalLengthCm"]]
y = data["Species"]

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y)

model = MLPClassifier(hidden_layer_sizes=(10,10), max_iter=1000)
model.fit(X_train, y_train)

print("Accuracy:", model.score(X_test, y_test))
|
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X.iloc[:,0], X.iloc[:,1], X.iloc[:,2], c=y)

ax.set_xlabel("Sepal Length")
ax.set_ylabel("Sepal Width")
ax.set_zlabel("Petal Length")

plt.title("3D Visualization (3 Classes)")
plt.show()
```

Output:-